Optimized Scaling Laws Exercise

Rahul Bir

In this experiment, I fit scaling laws for small Transformer models on a simple addition problem. I then optimize the forward pass and training step time by fusing the MLP layer with custom Pallas kernels.

Experiment

I train Transformer models with vanilla MHA as MQA / grouped MQA seems unnecessary for the scale of the exercise. The MLP blocks use SwiGLU and positional encodings are implemented with RoPE.

Pairs of up to three digit numbers are generated and converted into string representations forming an addition equation. Spaces are randomly added around numbers and operands, forcing the model to read and extract numbers rather than depending on fixed position.

For the dense scaling experiments, I train six models increasing in size logarithmically from 4.4K to 150K. I initially run preliminary runs seeking to find a lower bound and upper bound for the compute range. I do this because of the simple nature of the addition task. It's easy for models to reach 0 loss and runs where multiple models saturate the loss are not helpful towards the scaling law since it comes down to noise for which model size at that compute range is chosen as the optimum.

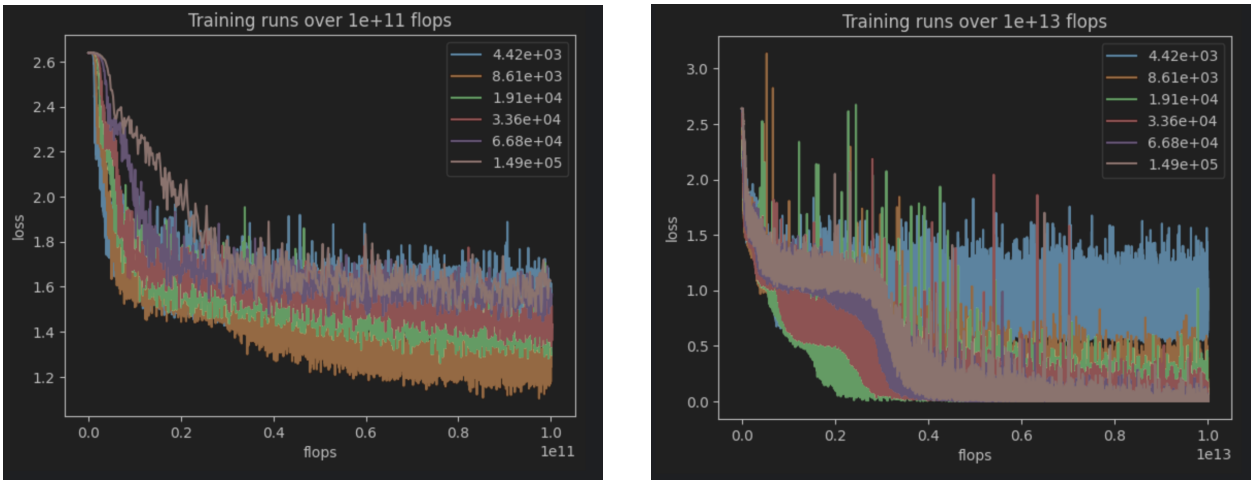

Figure 1: Preliminary sweeps used to bracket the compute range.

Each model is trained with the same learning rate at 1e-3 with a cosine decay of 10×, where 5% of the steps are used for warmup. This follows the Chinchilla paper's approach of using a separate learning rate decay for each run. I set only 5% of the steps as a warmup because at the lower range

of the compute grid the number of steps is already limited and I wanted to have two decades worth of compute measured. The bs is fixed at 32. I then gaussian smooth the loss curves of each grid to get rid of loss spikes. I empirically find using a sigma of size steps / 100 to give the best result and then set the radius of the kernel at 3σ.

I fit N_{opt} and D_{opt} using the three fitting approaches used in the Chinchilla paper:

1. Create interpolants from the last 15% of training curves and select optimum points from overlapping curves over a logarithmically spaced curve of compute points.
2. Fit parabolas of final loss v. model size and gather N_{opt} points by taking the min of these parabolas. Use optima to fit scaling laws.
3. Fit a parametric loss function with L-BFGS (using a grid of initializations for best fit) and the huber loss (which reduces the effects of outliers) and then derive equations for N_{opt} and D_{opt} based on the parameters of the fit loss function.

For MoE I repeat the experiment over the same compute range but base model size based on active parameters. I keep the sparsity fixed at 4 with 4 total experts for every config and 1 expert per token. The active model sizes are the same as in the dense version.

Results

For the dense models, approach 1 and 2 give an N_{opt} exponent of around 0. Inspecting some of the curves, this makes sense as the range in which the model size is the bottleneck and not able to saturate the loss is small due to the task's simplicity. After this point, the smallest model able to fit the addition circuit wins as it can process more tokens at the same compute. Approach 3 shows similar behavior though the scaling for N_{opt} is slightly higher. I attribute this due to the Huber Loss and the indirect calculations of N_{opt} and D_{opt}.

N_{opt} power

	Approach 1	Approach 2	Approach 3
Dense	−0.018	−0.039	0.154
MoE	0.105	0.990	0.251

D_{opt} power

	Approach 1	Approach 2	Approach 3
Dense	1.018	1.039	0.846
MoE	0.895	0.010	0.749

For the MoE models, approach 1 and 3 give nearly similar results to the dense models, but the

IsoFLOP approach 2 gives nearly opposite results — with N_{opt}'s exponent being close to 1. Looking at the isoflop curves, I attribute this to being a very poor fit of parabolas because of very shallow valleys. The IsoFLOPS approach is much more sensitive to noise at that scale than the other approaches.

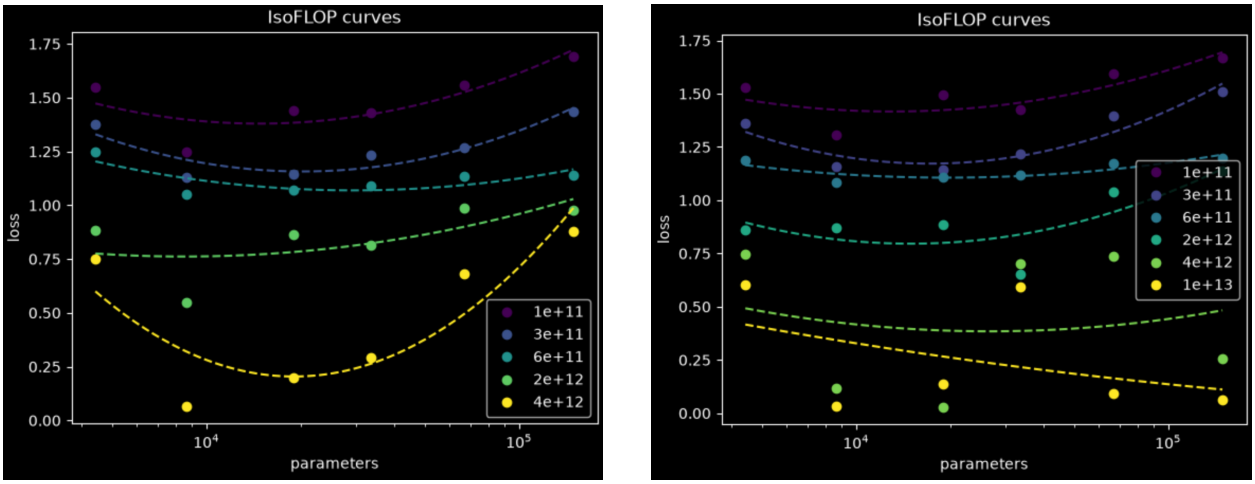

Figure 2: IsoFLOP parabola fits: dense (left) and MoE (right).

Forward + backward MLP layer kernel fusion optimization

The argument for the benefits of kernel fusion in the MLP is purely an arithmetic intensity one. In the forward pass, looking at the compiled HLO, XLA does not fuse or overlap either of the up projections with the gating and activation function. There are separate roundtrips each projection and the fused gating.

The AI then for the unfused version is

$$\text{AI}_{\text{fwd, unfused}} = \frac{6BDF}{4 \cdot (3BD + 6BF + 3EDF)}$$

whereas for the fused version it is

$$\text{AI}_{\text{fwd, fused}} = \frac{6BDF}{4 \cdot (3BD + 3EDF)}.$$

At $D = 256$, $F = 4096$, $E = 4$ with $B = 512 \cdot 17$, the AI of the unfused version is only 58 whereas the fused version is 710! The fused version is clearly compute bound and the fixed overhead from using a custom pallas kernel is clearly worth it. Under this setting, I get the fastest speedup of 1.7×. As I push D higher to 512, the unfused version starts to become compute bound so the speedups diminish.

In the backward pass, the AI becomes

$$\text{AI}_{\text{bwd, unfused}} = \frac{12BFD}{4 \cdot (11BF + 5BD + 6EDF)}.$$

Whereas the fused version gets rid of all the expensive BF intermediates with an AI of

$$\text{AI}_{\text{bwd, fused}} = \frac{16BFD}{4 \cdot (3BD + 6EDF)}.$$

At $D = 256$, $F = 4096$, the unfused backward pass has an AI of 64.5 whereas the fused version has an AI of 1146! Note, we have to recompute the forward intermediates in the fused backward pass since the fused forward does not save intermediate. However, this gives up to a 1.77× speedup for training step time.